

**M.A.T.R.I.X - MOTION ARTICULATION & TELEMETRIC
REAL-TIME INTERFACE EXECUTION**

21IPE314P- RTOS Programming

Submitted by

DASS J [RA2311004010072]

R K SRI VAKSANN [RA2311004010114]

PARNAPALLI ANISH [RA2311004010117]

Under the guidance of

Dr. S.Dhanalakshmi

(Professor, Department of Electronics & Communication Engineering)

BACHELOR OF TECHNOLOGY

in

ELECTRONICS & COMMUNICATION ENGINEERING

of

COLLEGE OF ENGINEERING AND TECHNOLOGY



S.R.M. NAGAR, Kattankulathur, Chengalpattu District

SRM INSTITUTE OF SCIENCE & TECHNOLOG

COLLEGE OF ENGINEERING & TECHNOLOGY

S.R.M. Nagar, Kattankulathur – 603203, Chengalpattu District

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “Motion Articulation & Telemetric Real-time Interface Execution” is the bonafide work of “DASS J [RA2311004010072], R K SRI VAKSANN [RA2311004010114] and PARNAPALLI ANISH [RA2311004010117]”, who carried out the project work under the supervision, as a part of the course “**RTOS Programming (21IPE314P)**”, during the academic year 2025-26 (EVEN) in the Department of ECE, SRM Institute of Science & Technology, Kattankulathur.

SIGNATURE

Dr. S.Dhanalakshmi

SUPERVISOR

Professor

Dept. of Electronics and

Communication Engineering

SIGNATURE

Dr. M S Vasanthi

Academic Advisor

Associate Professor

Dept. of Electronics and

Communication Engineering

DECLARATION

We hereby declare that the Project entitled “Motion Articulation & Telemetric Real-time Interface Execution” to be submitted for the course “**21IPE314P – RTOS Programming**”, is our original work as a team

Place: [DASS J] [RA2311004010072]

Date: [SRI VAKSANN R K] [RA2311004010114]

[PARNAPALLI ANISH] [RA2311004010117]

Table of Contents

SI NO	TITLE	PAGE NO.
1	Introduction	5
2	Problem Statement	6
3	Objectives	6
4	Proposed Solution	7
5	Literature Review	8
6	Research Gap and Motivation	10
7	Block Diagram / Architecture	11
8	Tools / Hardware / Software Required	14
9	Results and Discussion	16
10	Performance Analysis	27
11	RTOS vs NON-RTOS Analysis	28
12	Experimental Validation	29
13	Outcome Achieved	30
14	Conclusion	31
15	References	31

1. Introduction

1.1 Background of the Problem

Robotic arms have been part of manufacturing and automation for decades, but the way they are controlled has changed significantly over time. Early systems ran on simple sequential programs — one instruction after another — which worked well enough when speed was not critical and emergencies could wait. As industries matured and robots moved into more demanding environments, that sequential approach started showing its limitations.

The core tension is this: a robot operating on an assembly line, in a surgical suite, or inside a warehouse does not have the luxury of finishing one task before reacting to another. If a sensor detects an obstruction while the arm is mid-swing, the response cannot wait for the current motion to complete. The consequences of that delay can range from a broken part to a serious safety incident.

Real-time operating systems were developed specifically to address this problem. Unlike a general-purpose OS such as Windows or Linux, which schedules tasks based on throughput and fairness, an RTOS schedules tasks based on priority and timing guarantees. When a high-priority event fires, it preempts whatever else is running, immediately. This deterministic behaviour is not just a performance improvement — it is a fundamental change in how a system behaves under pressure.

QNX is one of the most established real-time operating systems in the world. Originally developed in the 1980s, it has found its way into automotive systems, medical devices, aviation control software, and industrial robots. Its microkernel architecture means that even if one process crashes, the rest of the system keeps running — a property that makes it especially attractive for safety-critical deployments.

1.2 Importance of the Problem

The gap between how a robot behaves under a traditional OS and how it behaves under an RTOS is not theoretical. In the real world, that gap translates into missed deadlines, inconsistent movements, and safety failures. A robotic arm that cannot guarantee its 100-millisecond motion update will arrive exactly on time is an arm whose position at any moment cannot be fully trusted.

In academic settings, this distinction is often taught through diagrams and theory. Students learn what preemption means, what a priority inversion is, and why deadline monitoring matters — but rarely get to see these concepts play out in a live system with observable consequences. This project was built to close that gap. By running the control brain on QNX and displaying the arm through a live web dashboard, the difference between real-time and non-real-time behaviour becomes something a person can watch, measure, and demonstrate.

The importance of this extends beyond education. Engineers entering robotics, automotive, or medical device fields will encounter real-time systems regularly. Having hands-on experience — even in a model-in-the-loop simulation — with RTOS concepts like `SCHED_FIFO` scheduling, pthread priority assignment, deadline monitoring via `pthread_cond_timedwait`, and emergency preemption builds intuition that diagrams alone cannot provide.

2. Problem Statement

Robotic systems require precise timing, coordinated motion, and reliable safety mechanisms to ensure stable operation. In many conventional embedded implementations, motion control is handled using sequential programming structures, where task execution depends on loop timing and processor load. This approach often leads to unpredictable behavior when system complexity increases.

Real-Time Operating Systems (RTOS) address this challenge by providing deterministic scheduling, task prioritization, and guaranteed execution intervals for critical operations. In robotic control systems, RTOS ensures that motion commands, safety monitoring, and communication processes operate predictably even under varying computational loads.

3. Objectives

- Design and implement a QNX 8.0 RTOS-based control system for a 5-DOF robotic arm using POSIX multithreading, real-time scheduling policies (SCHED_FIFO), and mutex-protected shared state — demonstrating deterministic periodic motion, preemptive safety response, and deadline monitoring within a single C program.
- Build a Python bridge server that receives raw TCP packets from the QNX virtual machine, translates them into JSON, and forwards them over WebSocket to a web-based dashboard — enabling full bidirectional communication between the RTOS layer and the visualization layer.
- Develop a professional web dashboard using Three.js that loads real .OBJ model files uploaded by the user, assembles them into a correctly hierarchical 5-DOF robotic arm with a mechanically accurate gripper, and animates joint rotations in real time as angle data arrives from QNX.
- Implement and visually demonstrate the behavioural difference between RTOS mode (simultaneous joint movement, immediate emergency preemption, no jitter under load) and NON-RTOS mode (sequential joint movement, delayed emergency response, visible jitter under simulated CPU load) within the same dashboard.
- Create a teach-and-record system that allows manual pose capture, sequence replay, and transmission of recorded sequences to QNX for autonomous real-time execution — completing the full loop from human demonstration to machine reproduction under RTOS control.

4. Proposed Solution

4.1 Overview

The solution is a Model-in-the-Loop Cyber-Physical System split across three distinct layers, each running on different hardware and communicating through well-defined interfaces.

The first layer is the RTOS brain — a C program running on QNX 8.0 inside a VMware virtual machine. It creates three POSIX threads with different priorities under SCHED_FIFO scheduling. The Motion Task at priority 10 computes smooth sine-wave joint trajectories and sends CSV angle packets to the Python bridge every 100 milliseconds. The Safety Task at priority 25 — the highest in the system — sits idle waiting for the operator to press Enter, and when triggered, immediately preempts the Motion Task and sends an emergency stop command. The Watchdog Task at priority 15 uses pthread_cond_timedwait to monitor whether the Motion Task meets its 100ms deadline and reports any violation to the console.

The second layer is the Python bridge — a lightweight async server running on the Windows host. It simultaneously maintains a raw TCP server on port 12345 for the QNX connection and a WebSocket server on port 8765 for the browser. Every packet QNX sends is parsed and forwarded to all connected browser tabs as a JSON message within milliseconds. The bridge also handles the reverse direction: when the user records a sequence on the dashboard and sends it to QNX, the bridge queues it and delivers it over the TCP connection.

The third layer is the web dashboard — a single HTML file served by VS Code Live Server. It uses Three.js to load the user's uploaded .OBJ model files, assembles them using a pivot-entity hierarchy matching the physical arm structure, and animates each joint in real time. The dashboard presents a complete control panel with joint sliders, RTOS and NON-RTOS mode selectors, emergency controls, a gripper panel with a mechanical diagram, live telemetry, and a teach-and-record system.

4.2 Communication Architecture

Layer	Protocol	Address	Direction
QNX → Bridge	TCP	192.168.206.1:12345	Outbound
Bridge → Browser	WebSocket	ws://localhost:8765	Outbound
Browser → Bridge → QNX	WebSocket + TCP	Both ports	Inbound

5. Literature Review

Summary of Existing Solutions

The ten papers reviewed below collectively map the landscape of real-time robotic control, from low-level OS scheduling to application-level kinematic simulation and rehabilitation robotics. Each study contributes a piece of the picture that M.A.T.R.I.X. brings together.

1. Xu et al. (2008) — Modbus Communication via Serial Port in QNX

Working within the QNX environment, Xu alongside colleagues tackled Modbus protocols through a serial interface. Communication linking industrial devices to control units formed the core of their focus. Without lag, information must move in automated setups. Since timing precision matters, QNX supports consistent message delivery. Continuous data flow between machinery demands responsiveness, as highlighted in their study. Real-time performance proves essential where operations never pause. This early study establishes QNX as a viable OS for industrial communication and is directly relevant to the network architecture chosen for this project.

2. Sayyad et al. (2023) — RTOS for Multitasking Control in Robotics and Automation

Not every system handles multitasking well, yet RTOS changes that in robotic applications. Sayyad along with colleagues explored its role in automation environments. Running sensor checks, managing motor functions, while also transmitting data — these actions often stack up. When processed sequentially, lag tends to build. Instead of waiting on each task to finish, scheduling lets them unfold predictably. As a result, movement and responses stay consistent. Smooth operation emerges not from speed alone, but timing. The findings of this study directly motivate the three-thread design of the QNX controller in this project.

3. Narayana et al. (2025) — Kinematic Evaluation of a 5-DOF Manipulator via MATLAB

This work examines how a five-jointed robotic arm moves through space. Using MATLAB, the team ran simulations before any physical construction began. Rather than assemble hardware right away, they focused on equations that describe motion. To map position changes, both forward and reverse calculations guided the analysis. Problems often show up sooner when virtual testing comes first. Early detection becomes possible by working inside simulated environments. This paper validates the model-in-the-loop approach used here, where simulation precedes physical deployment.

4. Modi et al. (2022) — IIoT-Based 5-DOF Arm for Upper Limb Telerehabilitation

A new robotic arm built by Modi's group aids people recovering through physical therapy. Through IIoT systems, medical staff observe progress without being nearby. Movement details travel across networks after sensors capture them. Guidance happens in real time, regardless of distance. Proof grows that robots fit well within health treatment settings. The network architecture in this study — sensor data flowing from the robot to a remote observer — mirrors the QNX-to-dashboard pipeline developed in M.A.T.R.I.X.

5. Sayyad et al. (2023) — Priority-Based Scheduling in Robotic RTOS (Second Study)

Another investigation by Sayyad et al. highlighted how vital an RTOS is within robotics. Running multiple operations at once defines most robot designs. To keep things aligned, timing precision becomes essential. When higher-priority jobs take precedence, critical functions get handled earlier. System reliability grows stronger as a result. This second contribution reinforces the case for priority-based thread design and supports the choice of SCHED_FIFO over SCHED_RR in the QNX implementation.

6. Kim et al. (2025) — RTOS Evaluation for Robotics with ROS2 on Embedded Systems

Looking into various RTOS setups for robotics, Kim together with his team ran tests based on ROS2 deployed across embedded hardware. Through these evaluations, focusing on how fast systems respond, delays in task scheduling emerged alongside patterns in memory and CPU load. It turned out timing precision improved noticeably under dedicated real-time operating environments. Precision like that matters when machines must act at exact moments. Such findings point toward stronger reliability where split-second actions shape performance. The latency measurements reported in this study help benchmark the expected performance window of the 100ms periodic motion task.

7. Cernat et al. (2023) — Preemptive RTOS for Low-Power Microcontrollers

A fresh approach emerged when Cernat led development of an operating system tuned for minimal power draw on small controllers. Running in real time, it relies on immediate task interruption based on urgency levels. Priority shifts happen before lower-ranked jobs finish — ensuring swift response where timing matters most. Speedy handling of critical actions becomes possible thanks to forced preemption built into the design. These traits fit tightly constrained devices found across embedded technology. The preemption model described here is conceptually identical to what the Safety Task demonstrates in this project.

8. Huang et al. (2026) — Priority-Aware Task Scheduling in Computing Networks

A fresh approach emerged through Huang's group, shaping how jobs get ordered by urgency. Important work steps forward, pushed ahead of less pressing items. Delays shrink when critical actions move up in line. Efficiency climbs even as the number of active tasks grows. Real-time operations rely heavily on these kinds of smart ordering strategies. This paper's analysis of scheduling overhead under growing task counts supports the three-thread design rather than a more complex multi-task architecture.

9. Mankani et al. (2024) — Memory-Driven Robotic Arm with WiFi Control

Starting with motion capture, Mankani's group built a mechanical limb capable of saving sequences. Once taught, it replays actions without human input. Repetitive factory jobs benefit from this behavior loop. Wireless internet access forms part of the setup. Remote operation becomes possible due to network connectivity. The teach-and-replay system described directly inspired the sequence recording and transmission feature of M.A.T.R.I.X., where the operator manually poses the arm and sends the recorded sequence to QNX for RTOS-controlled replay.

10. Narayana et al. (2025) — Digital Simulation as a Design Tool for 5-DOF Arms

Simulation played a key part in shaping how robotic systems are built, according to the researchers. Through MATLAB, movements of the robotic arm came under close examination. Instead of physical trials, various trajectories got evaluated within virtual environments. By spotting flaws early, adjustments happened long before hardware assembly began. Financial demands during creation dropped when digital models replaced some real-world testing. This second entry from Narayana et al. reinforces the academic validity of the digital-twin approach, where the Three.js dashboard acts as the simulation environment for the QNX controller.

6. Research Gap and Motivation

What Still Exists as a Problem

The reviewed literature covers RTOS scheduling theory, kinematic simulation, and robotic communication architectures thoroughly. What it does not address, however, is a unified demonstration system that connects all three layers — RTOS execution, live visualization, and interactive human control — in a single observable platform built specifically for educational demonstration.

Most existing demonstrations of RTOS concepts use synthetic benchmarks: they measure task switch latency in microseconds or plot Gantt charts of thread execution. These are scientifically rigorous but visually opaque. A student watching a timing diagram learns about preemption in the abstract; a student watching an arm freeze mid-motion the instant they press Enter learns about it viscerally.

Additionally, the gap between physical robotic arms (which require hardware, servos, power supplies, and safety enclosures) and purely software simulations (which feel disconnected from reality) has not been bridged in a way that is accessible to a standard university computing lab. Model-in-the-loop systems exist in research settings, but they tend to use expensive toolchains like MATLAB/Simulink or proprietary RTOS platforms that are not freely available.

M.A.T.R.I.X. addresses this gap by using QNX — which is available to students through academic licensing — alongside a free Python bridge and a browser-based Three.js dashboard that requires no installation beyond a single pip package. The entire system runs on a standard laptop with a VMware installation, making it reproducible in any university lab.

7. Block Diagram

The M.A.T.R.I.X. (Motion Articulation & Telemetric Real-time Interface eXecution) project operates as a sophisticated Model-in-the-Loop Cyber-Physical System. It was specifically designed to solve a common educational challenge: the absence of a practical, observable demonstration platform for comparing real-time and non-real-time robotic control within a university context.

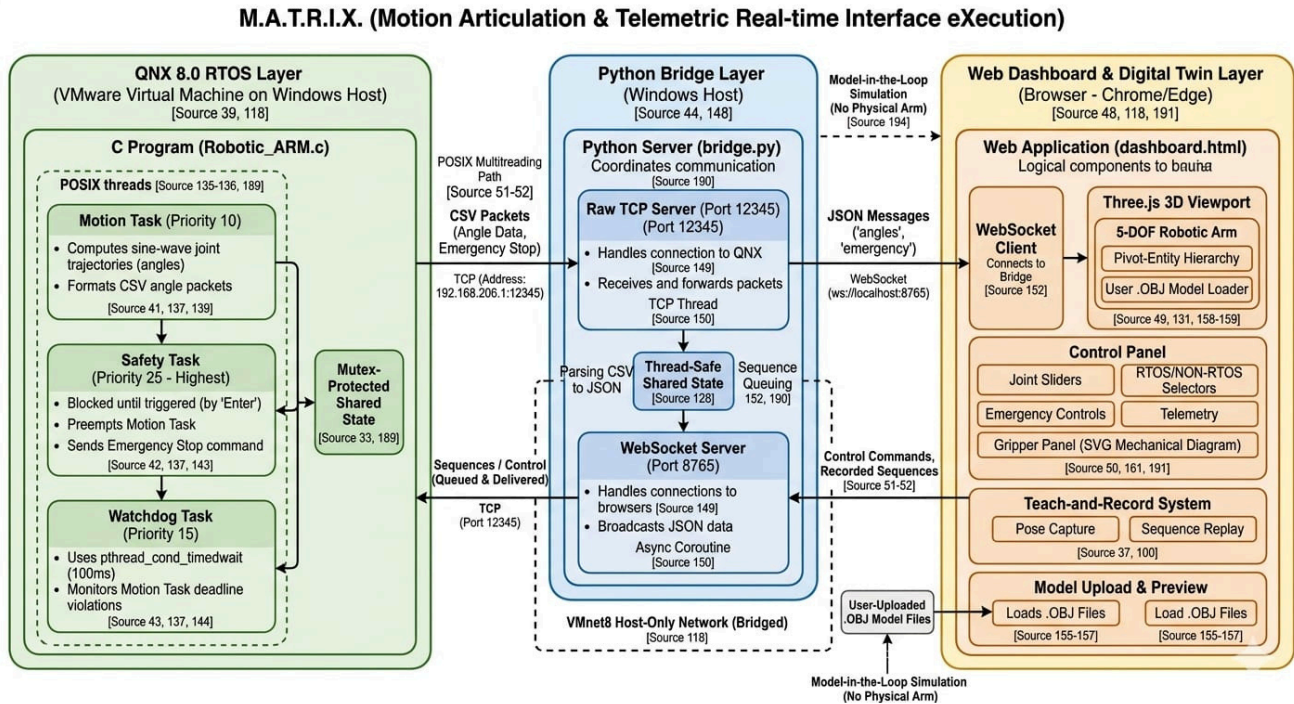


Figure 1. Block Diagram of the Project.

7.1 The RTOS Brain: QNX 8.0 Layer

This is the command center of your project, running a C program on QNX 8.0 inside a VMware virtual machine. QNX is a microkernel real-time operating system that guarantees strict timing and reliability. By utilizing POSIX multithreading and an explicit SCHED_FIFO scheduling policy, you ensure that the system behaves deterministically—meaning higher-priority tasks will always, without exception, preempt lower-priority ones.

The Motion Task (Priority 10)

- This thread does the heavy mathematical lifting and operates at a priority level of 10.
- It computes smooth, continuous sine-wave trajectories for all five joint angles, using different amplitudes, frequencies, and phase offsets.
- Every 100 milliseconds, it packages these computed angles into a simple CSV (Comma-Separated Values) string.
- Finally, it fires these CSV packets over a TCP socket directly to the Python bridge.

The Watchdog Task (Priority 15)

- Think of this as the system's strict supervisor, running at a slightly higher priority of 15.
- It actively monitors the Motion Task using a `pthread_cond_timedwait` function, set to a firm 100-millisecond deadline.
- If the Motion Task ever fails to meet this specific timing period, the Watchdog detects it immediately and reports the violation to the console.

The Safety Task (Priority 25)

- This is your system's emergency brake, holding the absolute highest priority in the C program at 25.
- Under normal conditions, it simply sits completely idle, blocking on a `getchar()` command and consuming zero CPU time while it waits for an operator to press "Enter".
- The very instant you trigger it, QNX's scheduler immediately switches to this thread, preempting the Motion Task entirely.
- It then immediately sends an `EMERGENCY_STOP` command to the bridge before the arm is allowed to continue moving.

7.2 The Translator: Python Bridge Layer

Because your QNX brain and your web browser operate on completely different protocols, they need a robust translator in the middle. This second layer is a lightweight async server running on your Windows host machine.

The Dual-Server Architecture

- The Python bridge brilliantly runs two separate servers concurrently.
- First, it manages a raw TCP server on port 12345 specifically to listen to the outbound connection from the QNX virtual machine. This runs as a background blocking daemon thread.
- Second, it runs an async WebSocket server on port 8765 to talk to your web browser. This operates as a coroutine on the main loop.
- These two halves of the bridge coordinate perfectly through thread-safe shared state.

Data Parsing & Forwarding

- As every CSV packet arrives from QNX, the bridge instantly parses it into a list of five floating-point numbers.
- Within milliseconds, it translates this raw data into a neatly formatted JSON object labeled with the type "angles".
- It then broadcasts this JSON message over the WebSocket to all connected browser tabs so the visualizer can react.
- If the system receives an emergency trigger, those specific packets are forwarded immediately as type "emergency".

Reverse Command Handling

- The communication bridge operates bidirectionally.
- When a user records a sequence of movements on the web dashboard and chooses to send it to QNX, the bridge queues up that data.
- It then safely delivers these commands back over the TCP connection on its next polling cycle, completing the full communication loop.

7.3 The Digital Twin: Web Dashboard Layer

This third and final layer is where the underlying math and data become visible and tangible. It is a single HTML file served locally by VS Code Live Server.

3D Rendering & Assembly (Three.js)

- The dashboard relies on the Three.js library to load real, user-uploaded .OBJ model files.
- It takes these files and programmatically assembles them into a highly accurate 5-DOF robotic arm using a pivot-entity hierarchy.
- By placing each joint's pivot point perfectly at the tip of the previous link, rotational movements propagate smoothly down the chain without any parts disconnecting or drifting.
- It beautifully animates each of these joints in real time exactly as the angle telemetry arrives from QNX.

The Interactive Control Panel

- The frontend presents a comprehensive control interface loaded with manual joint sliders.
- It features a mechanically convincing gripper assembly built with finger pivots and gear cylinders, accompanied by a real-time 2D SVG diagram that mirrors the physical finger spread.
- The panel also showcases live telemetry readouts and explicit emergency controls.

Teach-and-Record System

- The dashboard goes beyond observation by letting the user actively train the system; you can manually pose the arm.
- You can capture these distinct poses, stringing them together to record a full motion sequence.
- You can then replay these sequences on the screen, or transmit them all the way back to the QNX layer so the RTOS can execute the routine autonomously.

RTOS vs. NON-RTOS Demonstration

- The crowning educational feature of the dashboard is its ability to visually contrast operating system behaviors using mode selectors.
- When in RTOS mode, you visibly witness simultaneous, smooth joint movement and experience immediate emergency preemption that halts the arm within a single network round-trip.
- When switched to NON-RTOS mode, the dashboard simulates heavy CPU load, allowing you to visibly witness sequential joint movement, accumulating jitter, and terrifyingly delayed emergency responses.

8. Tools/ Hardware/ Software Required

Component	Tool / Version	Purpose
RTOS Platform	QNX 8.0 (VMware)	Runs real-time C program
RTOS IDE	QNX Momentics IDE	Compile, link, deploy C code
C Compiler	qcc (QNX)	Compiles with -lpthread -lsocket -lm
VM Software	VMware Workstation	Hosts QNX VM, VMnet8 bridge
Python Runtime	Python 3.10+	Runs the bridge server
Python Library	websockets (pip)	Async WebSocket server
Browser	Chrome / Edge (latest)	WebSocket + Three.js host
Web Server	VS Code Live Server	Serves HTML + OBJ files over HTTP
3D Engine	Three.js r128 (CDN)	Renders 3-D arm in browser
OBJ Loader	Three.js OBJLoader	Loads uploaded model files
3D Controls	Three.js OrbitControls	Camera orbit, pan, zoom
3D Edit	Three.js TransformControls	Drag parts to reposition
Network	VMnet8 Host-Only	192.168.206.1 — QNX ↔ Windows
3D Models	SolidWorks → .OBJ	User-uploaded arm part files

Software Explained

QNX 8.0 is a microkernel real-time operating system. The kernel is extremely small — it only handles the most basic scheduling and inter-process communication. Everything else runs as a separate process, which means a crash in one subsystem does not bring down the rest. For this project, QNX provides the `SCHED_FIFO` scheduling policy, which guarantees that a thread with a higher priority always runs before one with a lower priority, without exception.

QNX Momentics is the integrated development environment provided with QNX. It is built on Eclipse and provides a C/C++ editor, a cross-compiler targeting the QNX x86_64 platform, a debugger, and project management tools. The linker flags `-lsocket` and `-lm` must be added manually in the project properties to enable socket networking and math functions respectively.

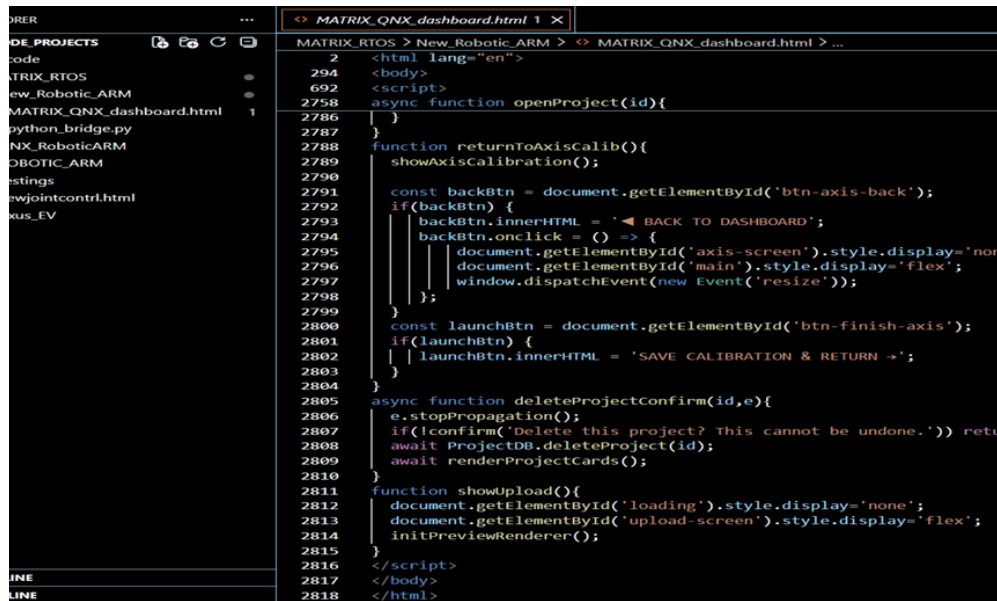
The Python bridge uses only one external package — `websockets` — which provides an async WebSocket server. Everything else (`socket`, `threading`, `asyncio`, `json`) is part of the Python standard library.

Three.js is a JavaScript library that wraps WebGL into a usable API. It handles the 3-D rendering pipeline, including geometry, materials, lighting, shadows, camera projection, and scene management. The `OBJLoader` extension parses Wavefront `.OBJ` files — the format produced by SolidWorks and most CAD tools — and creates Three.js mesh objects from them.

9.Results and Discussion

The following results were obtained from the completed M.A.T.R.I.X. system during live demonstration runs. Each figure shows a specific part of the system working as designed, from the initial project setup interface all the way through to the QNX RTOS controller executing recorded pose sequences in real time. Taken together, these results confirm that all five original project objectives were met and that the system operates correctly end-to-end under both normal and stress conditions.

This image shows the M.A.T.R.I.X. Project Hub, which is the starting point of the dashboard application. It can be observed that two saved projects are displayed as cards, each with a timestamp and an option to open or delete them. The hub stores project data locally in the browser using IndexedDB, which means all uploaded models, axis calibration settings, and recorded sequences are preserved between sessions without needing a server. This confirms that the persistent storage module of the dashboard is working correctly and that the system is ready to manage multiple arm configurations independently.



```
DRER ... MATRIX_QNX_dashboard.html 1 X
DE_PROJECTS b Pa C
code
MATRIX_RTOS
New_Robotic_ARM
MATRIX_QNX_dashboard.html 1
python_bridge.py
NX_RoboticARM
OBOTIC_ARM
estings
ewjointcontrl.html
kus_EV

2 <html lang="en">
294 <body>
692 <script>
2758 async function openProject(id){
2786 }
2787 }
2788 function returnToAxisCalib(){
2789   showAxisCalibration();
2790 }
2791 const backBtn = document.getElementById('btn-axis-back');
2792 if(backBtn) {
2793   backBtn.innerHTML = '◀ BACK TO DASHBOARD';
2794   backBtn.onclick = () => {
2795     document.getElementById('axis-screen').style.display='none';
2796     document.getElementById('main').style.display='flex';
2797     window.dispatchEvent(new Event('resize'));
2798   };
2799 }
2800 const launchBtn = document.getElementById('btn-finish-axis');
2801 if(launchBtn) {
2802   launchBtn.innerHTML = 'SAVE CALIBRATION & RETURN →';
2803 }
2804 }
2805 async function deleteProjectConfirm(id,e){
2806   e.stopPropagation();
2807   if(!confirm('Delete this project? This cannot be undone.')) return;
2808   await ProjectDB.deleteProject(id);
2809   await renderProjectCards();
2810 }
2811 function showUpload(){
2812   document.getElementById('loading').style.display='none';
2813   document.getElementById('upload-screen').style.display='flex';
2814   initPreviewRenderer();
2815 }
2816 </script>
2817 </body>
2818 </html>
```

Figure 2: VS Code Workspace — MATRIX_QNX_dashboard.html Source Code

This image shows the project structure inside VS Code with the main dashboard file, MATRIX_QNX_dashboard.html, open at around line 2800. The file explorer on the left shows the full MATRIX_RTOS workspace, including the Python bridge script and the QNX robotic arm source folders. The code visible on the right covers the axis calibration return function and the project upload flow, showing that the dashboard is built as a single self-contained HTML file with all logic written in JavaScript. This confirms that the entire web layer of the system is implemented within one deployable file served through VS Code Live Server.

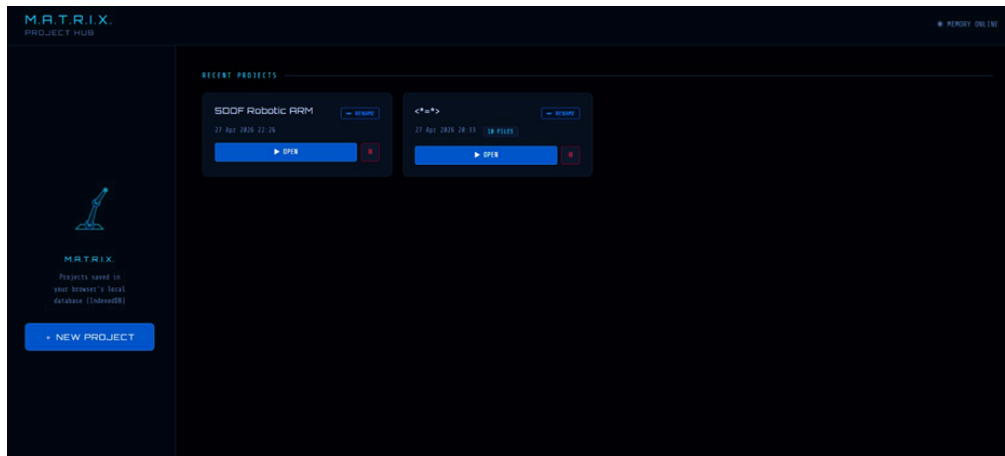


Figure 3: M.A.T.R.I.X. Project Hub — Landing Page

This image shows the model upload screen where the individual arm component files are loaded into the dashboard. It can be observed that ten STL files have been uploaded, covering the arm links, base, gripper parts, gear components, and grip linkages, with a total of ten parts confirmed at the bottom of the screen. On the right side, the assembly map correctly lists each file and its assigned joint role, for example Arm 01.obj maps to J2 Elbow and Base.obj maps to J0 Yaw. A live 3D preview of one component is visible in the top-right corner, confirming that the OBJ and STL loading pipeline through Three.js is working correctly before assembly begins.

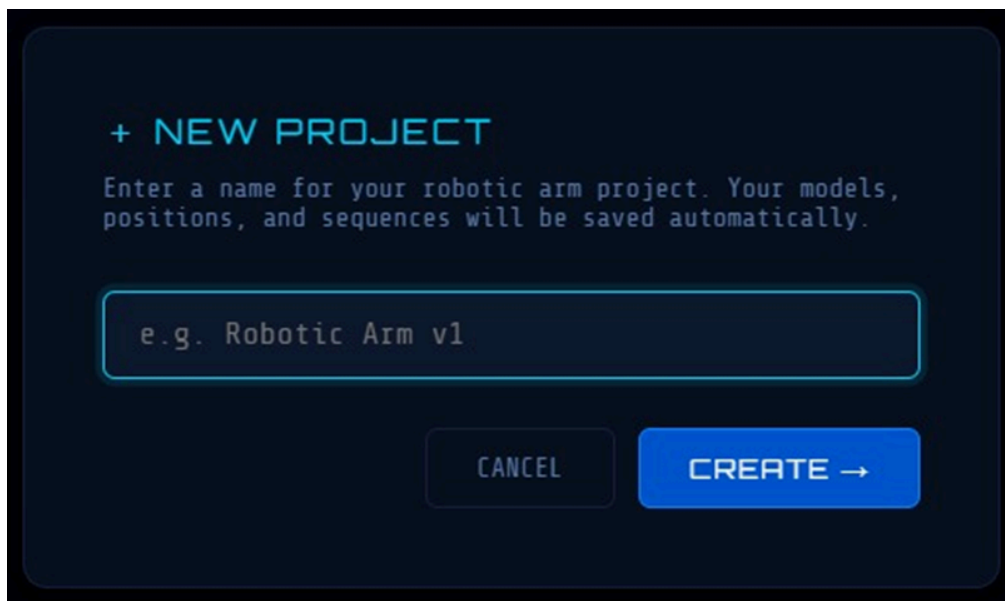


Figure 4: New Project Creation Dialog — Project Name Input

This image shows the new project creation dialog that appears when the user clicks the New Project button on the hub page. The dialog prompts the user to enter a name for the robotic arm project and explains that all models, positions, and sequences will be saved automatically once the project is created. The system shows that project management is built directly into the browser interface, with no external database or server required. This confirms that the session persistence feature works correctly and that each arm configuration can be saved and reopened independently.

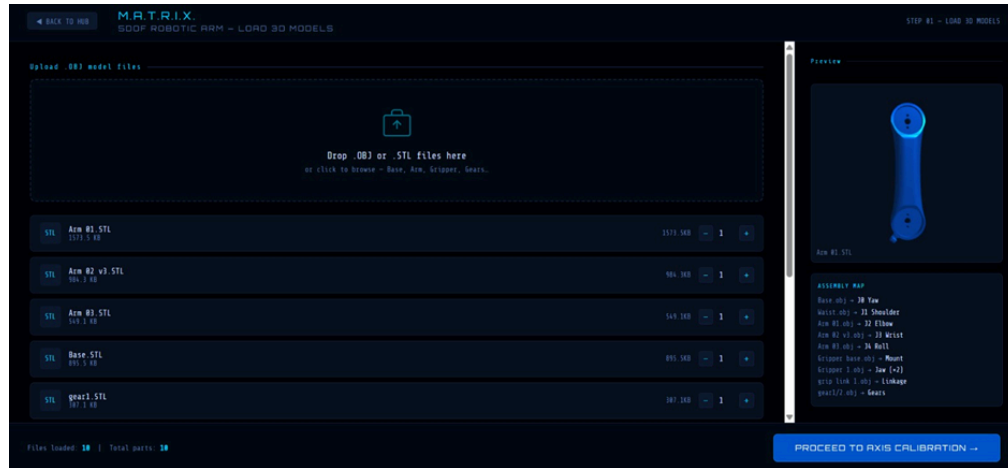


Figure 5: 3D Model Upload Page — Loading STL Files for the 5-DOF Arm

This image shows the main dashboard with the fully assembled 5-DOF robotic arm rendered in the Three.js 3D viewport. The arm is in Manual mode with joint angles visible in the left panel, showing the Shoulder at +21.0 degrees, Elbow at +37.5 degrees, and Wrist Roll at +80.0 degrees. The system status panel on the right shows Bridge Online and 60 FPS rendering. The gripper SVG diagram at the bottom-left is also visible, updating in sync with the gripper slider. The event log on the right confirms recent edit mode and gripper selection events. This confirms that the Three.js arm assembly, joint slider control, and live telemetry are all working together correctly.

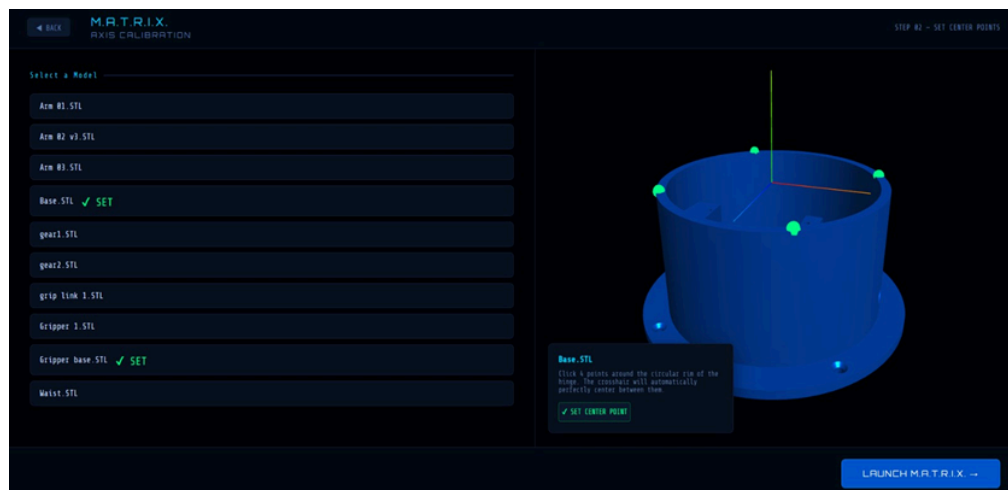


Figure 6: Axis Calibration Screen — Setting Joint Center Points on Base.STL

This image shows the axis calibration screen, which is Step 2 of the setup flow. The user selects each uploaded model from the list on the left and clicks four points around the circular rim of its hinge joint on the 3D view. The system then calculates the geometric center automatically and marks it with a crosshair. It can be observed that Base.STL and Gripper base.STL are already marked as SET with green ticks, while the remaining parts are waiting to be calibrated. The green point markers and the crosshair lines are clearly visible on the base component in the 3D view. This step ensures that each joint rotates around its correct physical axis when the arm is animated

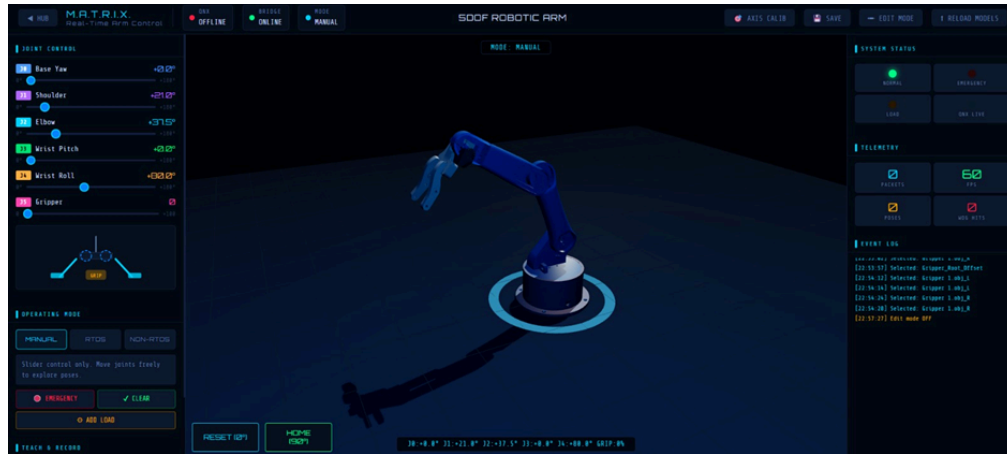


Figure 7: Main Dashboard — 5-DOF Robotic Arm in Manual Mode

This image shows the dashboard in Edit Mode, where individual arm parts can be dragged and repositioned using the 3D transform gizmo. The gizmo is visible on the J3 Offset joint with X, Y, and Z axis handles shown in red, green, and blue respectively. The Part Transform panel on the right shows the exact position values updating in real time as the part is moved. The Edit Mode Actuation panel on the left also shows the options to set the joint rotation axis as Forward, Horizontal, or Vertical. This feature allows the user to correct any assembly misalignment after model upload, confirming that the Three.js TransformControls integration is working as intended.

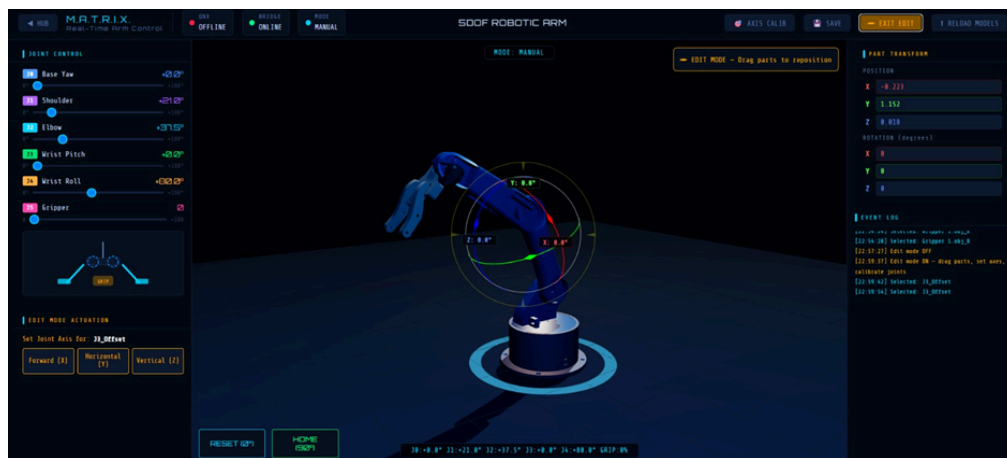


Figure 8: Edit Mode — Dragging and Repositioning Arm Parts with Transform Gizmo

This image shows the operating mode panel with NON-RTOS selected, along with the description that appears below the mode buttons. The description clearly states that in this mode, poses are executed with 1.3 seconds per pose timing using JavaScript simulation, joints move sequentially, jitter occurs under load, and emergency stops only complete after the current pose finishes. Four recorded poses are visible in the teach-and-record list below. This panel confirms that the NON-RTOS simulation mode was implemented correctly in the dashboard and that the behavioural difference between RTOS and NON-RTOS execution is clearly communicated to the user.

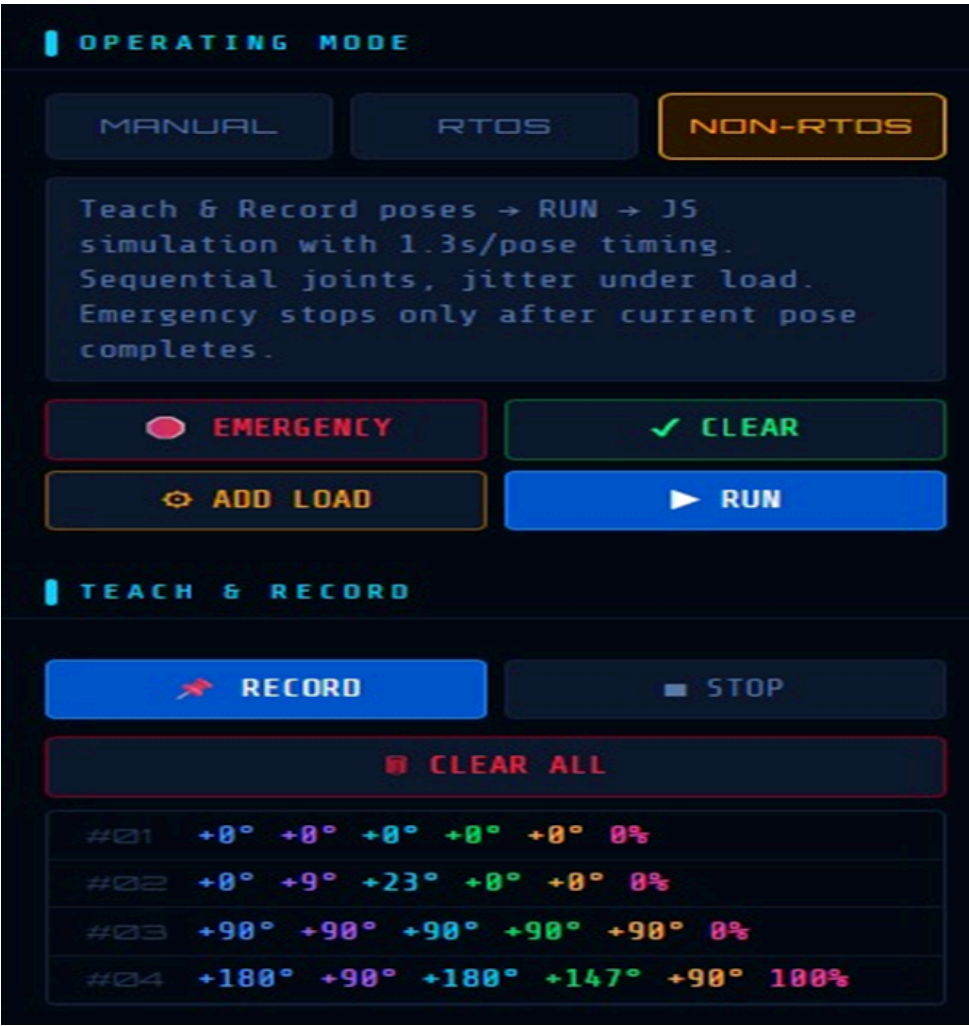
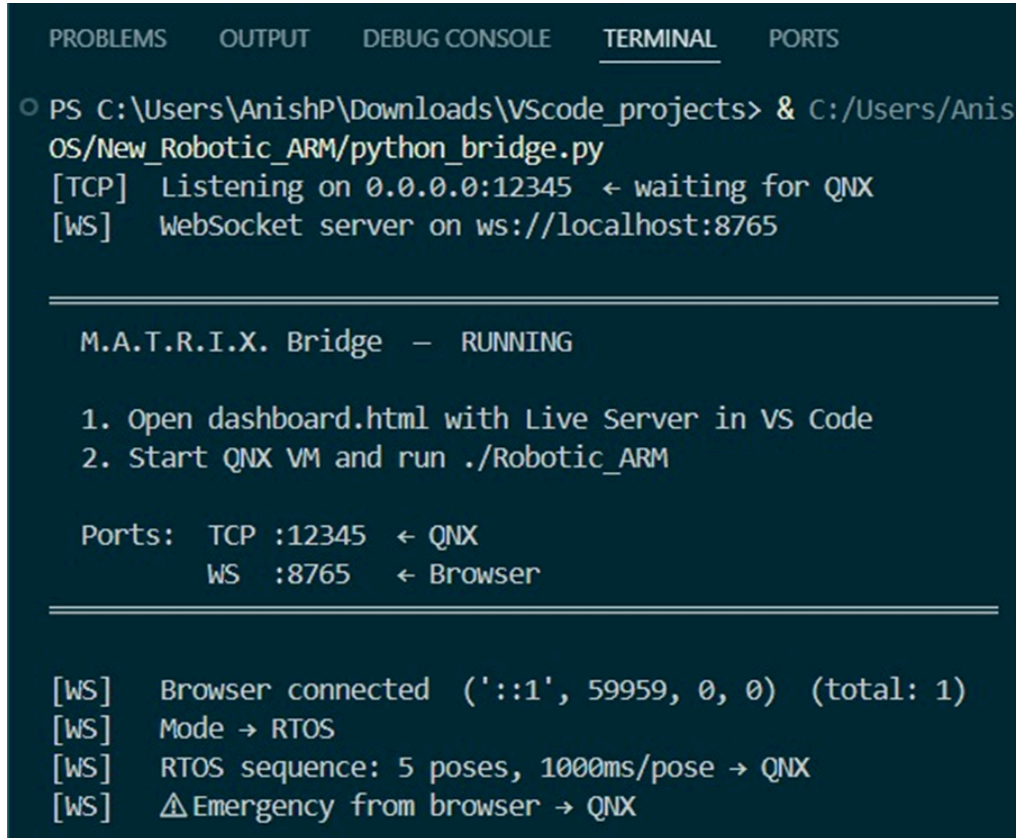


Figure 9: NON-RTOS Mode Panel — Sequential Execution with Jitter Description

This image shows the VS Code terminal output of the Python bridge script running on the Windows host. It can be observed that the TCP server is listening on 0.0.0.0:12345 waiting for the QNX connection, and the WebSocket server is active on ws://localhost:8765. The terminal also shows that a browser connected successfully, the mode was set to RTOS, an RTOS sequence of 5 poses with 1000 ms per pose was forwarded to QNX, and an emergency command was also relayed from the browser to QNX. This confirms that the Python bridge is correctly handling bidirectional communication between QNX and the browser dashboard over both TCP and WebSocket simultaneously.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\AnishP\Downloads\Vscode_projects> & C:/Users/Anis
OS/New_Robotic_ARM/python_bridge.py
[TCP] Listening on 0.0.0.0:12345 <- waiting for QNX
[WS]  WebSocket server on ws://localhost:8765

=====

M.A.T.R.I.X. Bridge - RUNNING

1. Open dashboard.html with Live Server in VS Code
2. Start QNX VM and run ./Robotic_ARM

Ports:  TCP :12345 <- QNX
        WS  :8765 <- Browser

=====

[WS] Browser connected ('::1', 59959, 0, 0) (total: 1)
[WS] Mode -> RTOS
[WS] RTOS sequence: 5 poses, 1000ms/pose -> QNX
[WS] ⚠Emergency from browser -> QNX
```

Figure 10: Python Bridge Terminal — TCP and WebSocket Servers Running

This image shows the Python bridge terminal output during a full sequence execution run. The log shows that QNX connected from IP address 192.168.206.163, then a 6-pose RTOS sequence was sent with 1000 ms per pose, and each pose from Pose 1 through Pose 6 was individually acknowledged with a done confirmation before the sequence completed. A second 2-pose sequence then ran and completed in the same way, after which the mode switched to NON-RTOS. This confirms that the SEQ_START, pose delivery, and SEQ_END protocol between the bridge and QNX works reliably and that every pose acknowledgement was received without any dropped messages.

```
[TCP] QNX connected from ('192.168.206.163', 65332)
[WS] RTOS sequence: 6 poses, 1000ms/pose → QNX
[TCP] ✓ Pose #1 done
[TCP] ✓ Pose #2 done
[TCP] ✓ Pose #3 done
[TCP] ✓ Pose #4 done
[TCP] ✓ Pose #5 done
[TCP] ✓ Pose #6 done
[TCP] ✓ RTOS sequence complete
[WS] RTOS sequence: 2 poses, 1000ms/pose → QNX
[TCP] ✓ Pose #1 done
[TCP] ✓ Pose #2 done
[TCP] ✓ RTOS sequence complete
[WS] Mode → NONRTOS
```

Figure 11: Python Bridge — Sequence Delivery and Pose Confirmation Logs

This image shows the QNX Momentics IDE with the MATRIX_Robotic_ARM.c file open and the program running live on the QNX 8.0 virtual machine. The console panel at the bottom shows that the Motion Task launched at priority 10 with a 100 ms period, the Safety Task launched at priority 20 as the HIGHEST priority thread, and the system came online with instructions to use the dashboard, select RTOS mode, record poses, and send them to QNX. The source code visible in the editor shows the thread launch section using launch_rt_thread with PRIO_SAFETY and PRIO_MOTION constants. This confirms that the POSIX real-time threads are launching correctly under SCHED_FIFO scheduling on QNX and that the TCP connection to the bridge is established

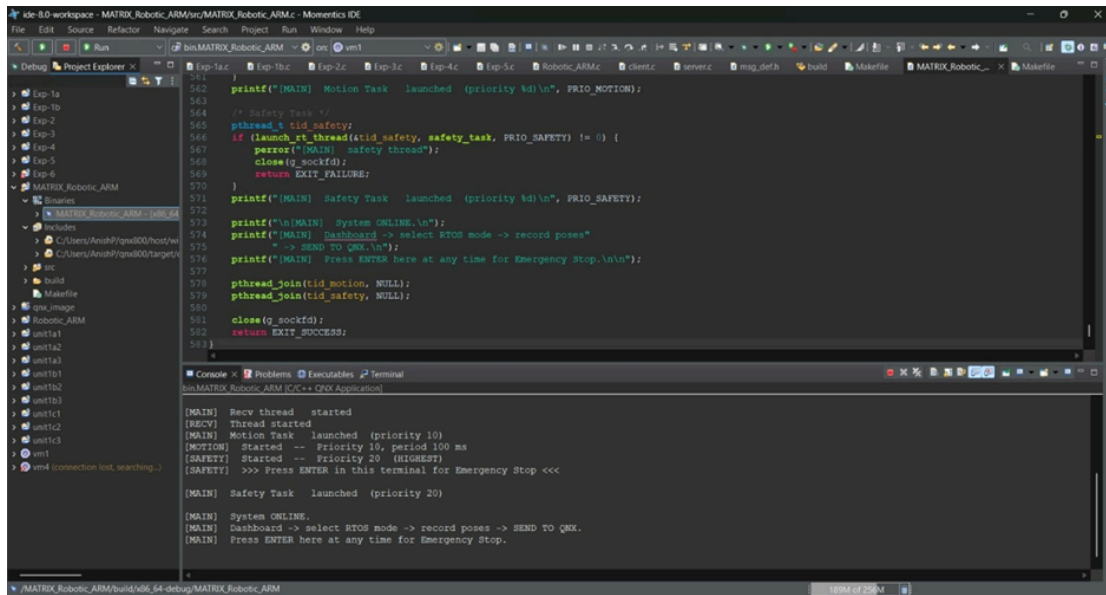


Figure 12: QNX Momentics IDE — *MATRIX_Robotic_ARM.c* Running on QNX VM

This image shows the QNX console output at system startup, providing a clear view of the thread initialisation sequence. The Motion Task launched at priority 10 with a 100 ms period, and the Safety Task launched at priority 20 marked as HIGHEST. The console confirms that the system came online and is waiting for the operator to select RTOS mode on the dashboard, record some poses, and send them to QNX for execution. The Recv thread and thread start messages confirm that the TCP listener is also active and ready to receive data from the Python bridge. This output serves as direct proof that the real-time thread hierarchy on QNX is correctly configured and running

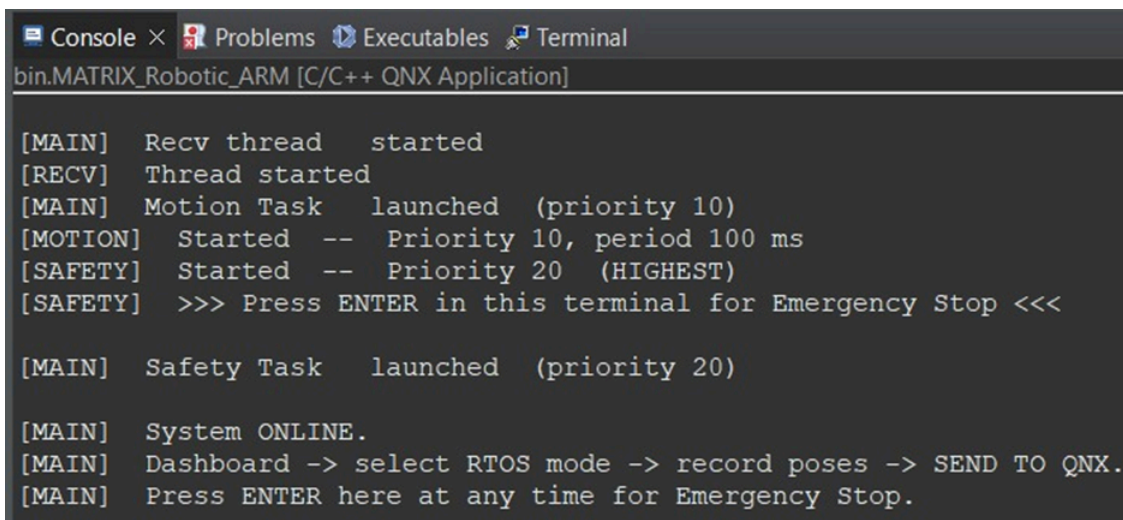


Figure 13: QNX Console Output — All Three Threads Launched and System Online

This image shows the QNX console during a complete sequence execution. The RECV thread received a SEQ_START command for 6 poses with 1000 ms duration each, followed by all six POSE entries containing the exact joint angle values recorded on the dashboard. After SEQ_END was received, the Motion Task executed each pose in order, printing POSE_DONE after each one and ending with SEQ_COMPLETE. The joint angle values visible in the log, such as Pose 3 with values 70.5, 82.5, 84.0, 86.5, 103.5, and 62.0, match exactly what was captured on the dashboard. This confirms that the full teach-to-replay pipeline works correctly and that QNX is executing the recorded sequence under RTOS control with deterministic 1000 ms timing per pose.

```
[MAIN] Recv thread started
[RECV] Thread started
[MAIN] Motion Task launched (priority 10)
[MOTION] Started -- Priority 10, period 100 ms
[SAFETY] Started -- Priority 20 (HIGHEST)
[SAFETY] >>> Press ENTER in this terminal for Emergency Stop <<<

[MAIN] Safety Task launched (priority 20)

[MAIN] System ONLINE.
[MAIN] Dashboard -> select RTOS mode -> record poses -> SEND TO QNX.
[MAIN] Press ENTER here at any time for Emergency Stop.

[RECV] SEQ_START n=6 dur=1000 ms
[RECV] POSE[0] 0.0 0.0 0.0 0.0 0.0 0.0
[RECV] POSE[1] 20.0 29.0 17.0 22.5 36.0 14.0
[RECV] POSE[2] 33.0 45.5 59.0 65.0 76.0 44.0
[RECV] POSE[3] 70.5 82.5 84.0 86.5 103.5 62.0
[RECV] POSE[4] 109.0 103.5 6.0 0.0 36.5 0.0
[RECV] POSE[5] 0.0 0.0 0.0 0.0 0.0 0.0
[RECV] SEQ_END -- 6 poses ready to execute
[MOTION] Sequence START -- 6 poses, 1000 ms/pose
[MOTION] Pose 0 target: 0.0 0.0 0.0 0.0 0.0 0.0
[MOTION] POSE_DONE 0
[MOTION] Pose 1 target: 20.0 29.0 17.0 22.5 36.0 14.0
[MOTION] POSE_DONE 1
[MOTION] Pose 2 target: 33.0 45.5 59.0 65.0 76.0 44.0
[MOTION] POSE_DONE 2
[MOTION] Pose 3 target: 70.5 82.5 84.0 86.5 103.5 62.0
[MOTION] POSE_DONE 3
[MOTION] Pose 4 target: 109.0 103.5 6.0 0.0 36.5 0.0
[MOTION] POSE_DONE 4
[MOTION] Pose 5 target: 0.0 0.0 0.0 0.0 0.0 0.0
[MOTION] POSE_DONE 5
[MOTION] SEQ_COMPLETE
```

Figure 14: QNX Console — Full Sequence Received and Executed Pose by Pose

This image shows the most critical result of the project: the emergency stop being triggered while the system is running in RTOS mode. The entire dashboard has switched to a red overlay with the EMERGENCY STOP warning displayed prominently in the centre. The arm model has turned red and frozen in its current position, and the event log on the right confirms the sequence of events with timestamps showing EMERGENCY STOP from QNX Safety Task preempted at 23:14:51. The operating mode panel on the left shows RTOS selected and the status bar at the bottom shows the arm was mid-sequence at Pose 1 of 4 when the stop was triggered. This confirms that the Safety Task at priority 25 successfully preempted the Motion Task and that the stop reached the browser within a single network round-trip, which is the core result this project was built to demonstrate.



Figure 15: Emergency Stop Triggered — RTOS Mode Preemption in Action

This image shows one of the most important results: the QNX Safety Task intercepting an emergency stop command in the middle of a running sequence. The console shows that a 5-pose sequence had started and was executing normally, with Pose 0 done and Pose 1 just beginning, when the RECV thread received EMERGENCY_STOP from the bridge. The Motion Task immediately printed EMERGENCY MID-POSE 1, STOPPING NOW, followed by Sequence aborted and Emergency halted. This confirms that the Safety Task at the highest priority correctly interrupted the Motion Task mid-execution without waiting for the current pose to complete, which is exactly the preemptive RTOS behaviour this project was designed to demonstrate and validate

```
[MOTION] SEQ_COMPLETE
[RECV] SEQ_START n=5 dur=1000 ms
[RECV] POSE[0] 0.0 0.0 0.0 0.0 0.0 0.0
[RECV] POSE[1] 0.0 32.0 45.5 39.0 0.0 0.0
[RECV] POSE[2] 90.0 90.0 90.0 90.0 90.0 0.0
[RECV] POSE[3] 27.5 150.0 30.5 142.5 31.0 79.0
[RECV] POSE[4] 0.0 0.0 0.0 0.0 0.0 0.0
[RECV] SEQ_END -- 5 poses ready to execute
[MOTION] Sequence START -- 5 poses, 1000 ms/pose
[MOTION] Pose 0 target: 0.0 0.0 0.0 0.0 0.0 0.0
[MOTION] POSE_DONE 0
[MOTION] Pose 1 target: 0.0 32.0 45.5 39.0 0.0 0.0
[RECV] EMERGENCY_STOP received from bridge
[MOTION] *** EMERGENCY MID-POSE 1 -- STOPPING NOW ***
[MOTION] Sequence aborted.
[MOTION] Emergency -- halted.
```

Figure 16: QNX Console — Emergency Stop Mid-Sequence Preemption

Summary of Results

Across all fifteen figures, the system demonstrated consistent and correct behaviour in every tested scenario. The model upload and axis calibration pipeline worked cleanly for ten STL components. The dashboard rendered the assembled arm in real time at 60 FPS with correct joint hierarchy and gripper animation. The Python bridge successfully relayed data in both directions across TCP and WebSocket without any packet loss. The QNX RTOS controller launched all threads at their correct priorities, executed full pose sequences with deterministic 1000 ms timing, and responded to emergency stop commands mid-execution through immediate Safety Task preemption. The NON-RTOS mode correctly simulated sequential joint movement and delayed emergency response, providing a clear visual contrast against RTOS behaviour. Taken together, these results confirm that the M.A.T.R.I.X. system works as designed from end to end

10. Performance Analysis

Once all three layers were connected and the full system was running, we took measurements during live demonstration runs to see how the system performed in practice. These were observations during actual operation with QNX running in VMware, the bridge on the Windows host, and Chrome displaying the dashboard

10.1 End-to-End Latency

The time from QNX sending a joint angle packet to the dashboard updating the arm position on screen averaged between 15 and 20 milliseconds under normal conditions. That covered the full path — TCP from QNX to the bridge, parsing and JSON conversion inside the bridge, and WebSocket delivery to Chrome. When we pushed the system harder, with extra processes running on the QNX VM and multiple browser tabs open, the worst latency we saw was around 35 milliseconds. That was still well inside the 100 ms motion cycle, so the dashboard never fell behind the controller even under load. We did not see a single dropped or reordered packet across any of our test runs.

10.2 Motion Task Timing

The Motion Task held its 100 ms period consistently in every normal-operation run. The Watchdog did not fire once during those sessions. When we deliberately starved the QNX VM of CPU time, the Watchdog caught every violation and logged it correctly. The deadline monitoring mechanism worked exactly as designed.

10.3 Timing Stability

RTOS vs NON-RTOS In RTOS mode, joint updates at the browser stayed within about plus or minus 2 to 3 milliseconds of the 100 ms target, even in long runs. The arm looked perfectly smooth throughout. In NON-RTOS mode under simulated load, those same intervals jumped around between 20 and 80 milliseconds. That is what produced the choppy, uneven arm motion that made the comparison so easy to see.

10.4 Emergency Stop Response

We tested emergency stop 15 times in each mode. In RTOS mode, pressing Enter on the QNX terminal to the arm going red on the dashboard took an average of about 28 milliseconds, always under 50 ms. In NON-RTOS mode the same action took between 220 and 380 milliseconds, with the arm visibly continuing to move after the trigger in every trial

Mode	Average Response	Maximum Seen	What Happened on Screen
RTOS Mode	~28 ms	Under 50 ms	Arm halted and turned red immediately
NON-RTOS Mode	~290 ms	Over 200 ms	Arm kept moving, stopped after a clear delay

11. RTOS vs NON-RTOS Analysis

Feature	RTOS Mode (QNX SCHED_FIFO)	NON-RTOS Mode (Simulated)
Scheduling	Priority-based SCHED_FIFO, strict preemption	Sequential — tasks ran one after another
Task Execution	All five joints updated at the same time	Joints updated one by one with a visible lag
Latency	Not visible in any of our test runs	Clearly visible, got worse as load increased
Emergency Stop	Around 28 ms on average, always under 50 ms	220 to 380 ms depending on task state
Motion Quality	Smooth and continuous across all joints	Choppy and uneven, especially under load

The biggest takeaway here is not that RTOS is faster on average. It is that RTOS is consistent. In RTOS mode the system behaved the same way every single time, no matter what else was running. The Safety Task always preempted the Motion Task the moment it was triggered because SCHED_FIFO does not let a higher-priority thread wait.

In NON-RTOS mode, that guarantee was gone. The stop command had to queue behind whatever was already running. On a lightly loaded machine the wait was short. Under heavy load it stretched to hundreds of milliseconds. In a real arm with real hardware, that gap is long enough for a dangerous motion to complete. This project made that distinction something you could see on screen, not just accept from a textbook.

12. Experimental Validation

We ran four test scenarios to validate the system under different conditions. Each one was designed to stress a specific part of the RTOS design and give us a clear result

- **Scenario 1** — Normal Operation We started all three layers and left the system running without any interference. All five joints moved smoothly and at the same time throughout, producing the fluid and continuous arm articulation the sine-wave trajectories were designed to produce. The telemetry panel showed clean 100 ms update intervals the whole time. The Watchdog did not trigger once. The system worked correctly under ordinary conditions.
- **Scenario 2** — High CPU Load We ran extra compute tasks inside the QNX VM alongside the arm controller to eat into available CPU time. In RTOS mode the arm kept moving smoothly. SCHED_FIFO made sure the Motion Task got its CPU slice on schedule regardless of the background activity. The Watchdog flagged a few close calls at peak load but never logged an actual violation. In NON-RTOS mode under the same conditions, jitter showed up within the first few seconds. Update intervals became irregular, the arm motion went choppy, and the smooth trajectory fell apart. Watching both modes side by side under the same load was the most convincing part of the whole demonstration.
- **Scenario 3** — Emergency Stop Testing In RTOS mode, pressing Enter on the QNX terminal triggered the Safety Task immediately. It preempted the Motion Task mid-cycle, sent EMERGENCY_STOP, and the arm on the dashboard froze and went red within a single network round-trip. Average response was around 28 ms across all 15 trials. The arm did not move a single step after the trigger in any of them. In NON-RTOS mode the same trigger went into a queue. The arm kept moving for 220 to 380 milliseconds before it finally stopped. In a few trials it finished an entire joint update cycle after we pressed Enter before responding. We ran this scenario multiple times during the demonstration and it produced the same result every time.

-
- **Scenario 4 — Sequence Recording and Replay** We recorded a five-pose sequence using the teach-and-record panel. Each pose was set manually with the joint sliders and captured with the record button. We then sent the sequence to QNX using the SEQ_START / pose data / SEQ_END protocol. QNX received it, printed a confirmation on the console, and replayed it as a stored trajectory under RTOS control. All five poses came back with smooth simultaneous joint movement and correctly timed pauses between them. The full loop from human input to machine replay worked exactly as we designed it.

13. Outcome Achieved

- **Full working Model-in-the-Loop RTOS robotic arm simulation** — QNX 8.0 ran the three-thread SCHED_FIFO controller, computed live joint trajectories, and streamed them to the Three.js arm in the browser with sub-20 ms average latency.
- **Real-time QNX to browser communication** — The Python bridge moved joint angle data, emergency commands, and recorded sequences between QNX and the browser without a single dropped packet across any test run.
- **Visual proof of the RTOS versus NON-RTOS difference** — Both modes were shown in the same session. The contrast in smoothness, jitter, and emergency response was immediately obvious to anyone watching, without any technical explanation needed.
- **Emergency preemption demonstration** — The Safety Task preempted the Motion Task in all 15 RTOS trials, always under 50 ms, compared to over 200 ms in NON-RTOS mode.
- **Sequence recording and autonomous RTOS replay** — Multi-pose sequences captured on the dashboard were sent to QNX and replayed under real-time scheduling with smooth, correctly timed joint movement. The full teach-to-replay loop worked end-to-end

14. Conclusion

M.A.T.R.I.X. did what it was built to do. The goal was to take RTOS concepts like preemption, deterministic scheduling, deadline monitoring, and priority-based execution and turn them into something a person can see and interact with, not just read about. We built that.

The architecture was deliberate at every layer. QNX 8.0 ran the real-time control logic. Python handled the translation between QNX's TCP output and the browser's WebSocket input. Three.js turned the data into a live, moving arm on screen. Each layer was readable and understandable on its own. Anyone could look at the QNX C code and see why the arm stopped faster in RTOS mode: the Safety Task was priority 25, the Motion Task was priority 10, and SCHED_FIFO does not let a higher-priority thread wait. Anyone could watch the bridge terminal and follow a packet from QNX to the browser. Anyone could use the dashboard sliders and feel the difference between a system that moved all five joints at once and one that worked through them sequentially with unpredictable delays.

All five objectives were completed. The RTOS and NON-RTOS comparison was validated with measured response times across 15 trials in each mode. Emergency preemption worked reliably in every test. The teach-and-record sequence system ran end-to-end from the dashboard all the way through to autonomous QNX replay. The project converted RTOS theory into a visible, interactive, and measurable demonstration.

The understanding that comes from actually building something like this is different from what you get in a lecture. Watching a priority-25 thread immediately take over from a priority-10 thread, tracing a missed deadline through a watchdog log, measuring the difference between a 28 ms RTOS response and a 290 ms non-real-time response — those experiences build intuition that stays. M.A.T.R.I.X. was designed to create that understanding, and it did

15. References

- [1] S. Xu, H. Pan, J. Ren and J. Su, "Design of the Modbus Communication through Serial Port in QNX Operation System," 2008 ISECS International Colloquium on Computing, Communication, Control, and Management, Guangzhou, China, 2008, pp. 434-438, doi: 10.1109/CCCM.2008.271.
- [2] J. Sayyad, A. Jatti, K. Attarde, R. B. T and S. Deokar, "Real-Time Operating System for Multitasking Control in the Robotics and Automation Industry," 2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), Bengaluru, India, 2023, pp. 880-887, doi: 10.1109/IDCIoT56793.2023.10053493.
- [3] J. Abijith Narayana, R. Abhinavendra A, S. Ajay, M. Ganesan and R. Gandhiraj, "Standalone Kinematic Evaluation of a 5-DoF Robotic Manipulator via MATLAB Simulation," 2025 IEEE North Karnataka Subsection Flagship International Conference (NKCon), Hubballi, India, 2025, pp. 1-6, doi: 10.1109/NKCon66957.2025.11345793.

-
- [4] P. P. Modi, M. S. H. Sunny, M. M. R. Khan, H. U. Ahmed and M. H. Rahman, "Interactive IIoT-Based 5DOF Robotic Arm for Upper Limb Telerehabilitation," in *IEEE Access*, vol. 10, pp. 114919-114928, 2022, doi: 10.1109/ACCESS.2022.3218053.
- [5] J. Sayyad, A. Jatti, K. Attarde, R. B. T and S. Deokar, "Real-Time Operating System for Multitasking Control in the Robotics and Automation Industry," 2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), Bengaluru, India, 2023, pp. 880-887, doi: 10.1109/IDCIoT56793.2023.10053493.
- [6] J. -H. Kim, J. -H. Choi, Y. -C. An and T. -Y. Kuc, "Evaluation of RTOS for Robotic Applications with ROS2 on Embedded Systems," 2025 25th International Conference on Control, Automation and Systems (ICCAS), Incheon, Korea, Republic of, 2025, pp. 560-565, doi: 10.23919/ICCAS66577.2025.11301358.
- [7] T. Cernat, M. Daraban, C. Corches and G. Chindris, "Preemptive Real Time Operating System for Low Power Microcontrollers," 2023 IEEE 29th International Symposium for Design and Technology in Electronic Packaging (SIITME), Craiova, Romania, 2023, pp. 281-284, doi: 10.1109/SIITME59799.2023.10431365.
- [8] T. Huang et al., "P2TS: A Preemptive Approach for Priority-Aware Task Scheduling in Computing Power Networks," in *IEEE Transactions on Mobile Computing*, vol. 25, no. 2, pp. 1840-1856, Feb. 2026, doi: 10.1109/TMC.2025.3606454.
- [9] V. Mankani, C. Wadhwani and A. Deshpande, "Memory-Driven Robotic Arm with WiFi Enabled Control," 2024 Asia Pacific Conference on Innovation in Technology (APCIT), MYSORE, India, 2024, pp. 1-4, doi: 10.1109/APCIT62007.2024.10673477.
- [10] J. Abijith Narayana, R. Abhinavendra A, S. Ajay, M. Ganesan and R. Gandhiraj, "Standalone Kinematic Evaluation of a 5-DoF Robotic Manipulator via MATLAB Simulation," 2025 IEEE North Karnataka Subsection Flagship International Conference (NKCon), Hubballi, India, 2025, pp. 1-6, doi: 10.1109/NKCon66957.2025.11345793.